

第十一周实验报告

实验 1——一位全加器

功能要求/工作原理:

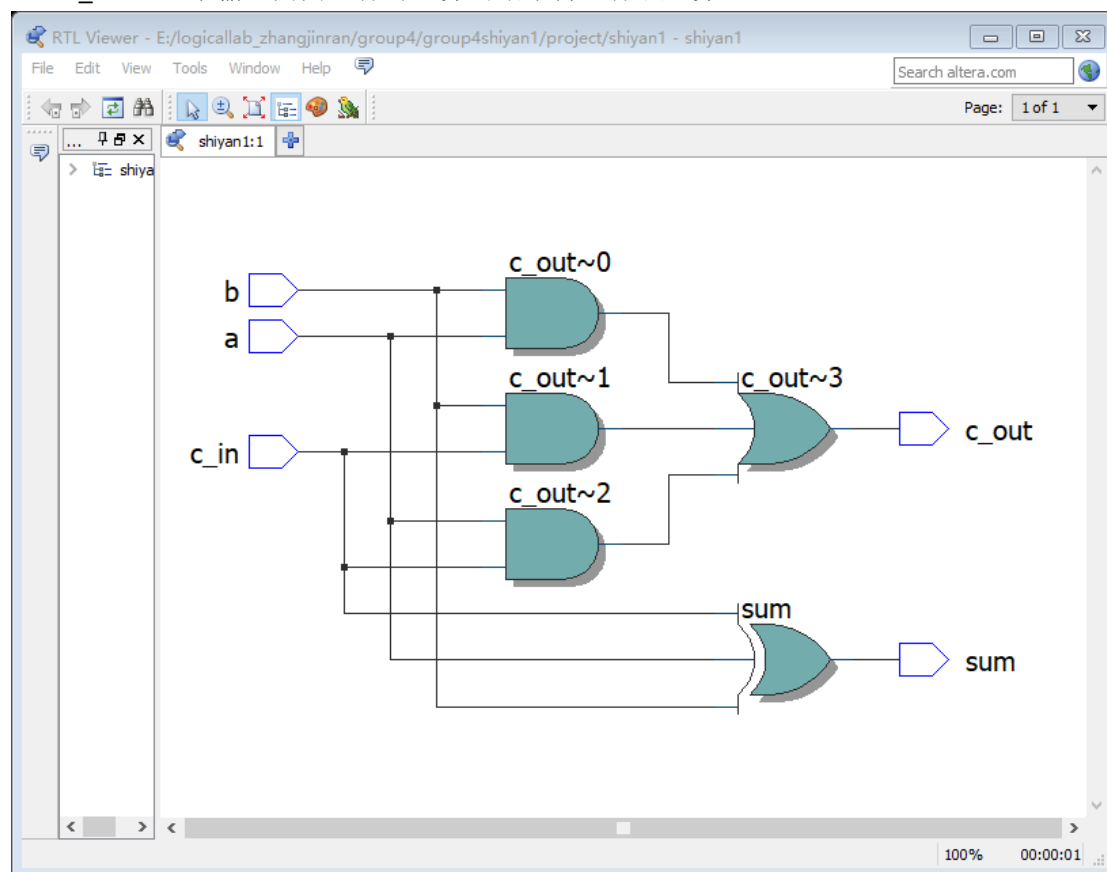
实现一位全加器。有加数 a、加数 b、低位进位输入 c_in 进行数据的输入，进行 a 和 b 的加法，同时考虑低位对它的进位，相当于三个数进行加法运算。输出的结果分为和 sum 和本位的进位输出 c_out。sum 为三个输入的异或 $\text{sum} = a \oplus b \oplus c_{in}$ ，c_out 为多数表决算法， $\text{c_out} = ab + bc_{in} + ac_{in}$ 。

调试步骤及过程说明:

三个输入，两个输出，三个二输入与门，一个异或门，一个三输入或门。

sum: 三个输入一起进行异或运算

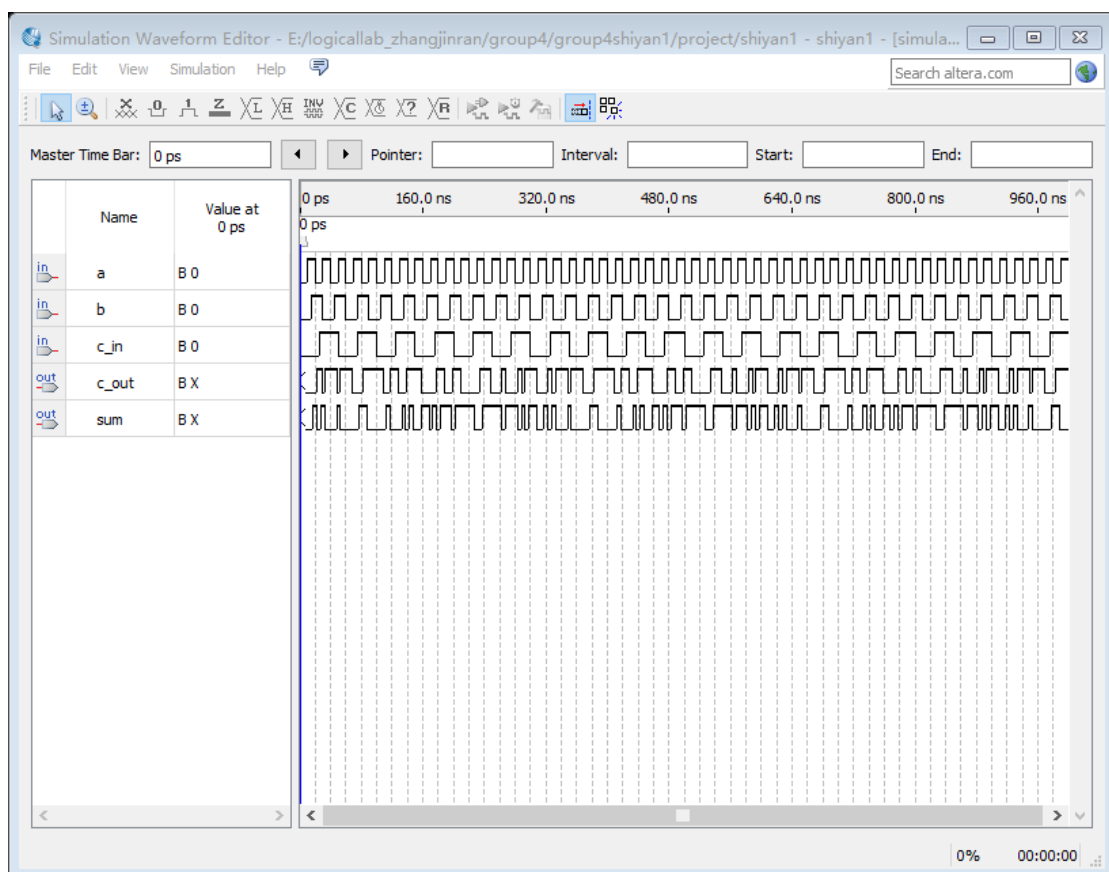
c_out: 三个输入两两进行与运算的结果再进行或运算



代码清单：

```
module shiyan1(  
    input wire a,  
    input wire b,  
    input wire c_in,  
    output wire sum,  
    output wire c_out  
);  
assign sum = (a ^ b) ^ c_in;  
assign c_out = (a & b) | (b & c_in) | (a & c_in);  
  
endmodule
```

仿真结果截图：



实验结果分析及其他问题说明：

sum 实现了三个输入的异或运算，实现了三个输入的奇偶检验。当三个输入中有奇数个输入为 1 时，sum 值为 1；否则，sum 的值为 0。

c_in 实现了三个输入中，有两个为 1 时，输出即为 1。

问题：时间延迟现象。

实验 2——3 选 2 表决器

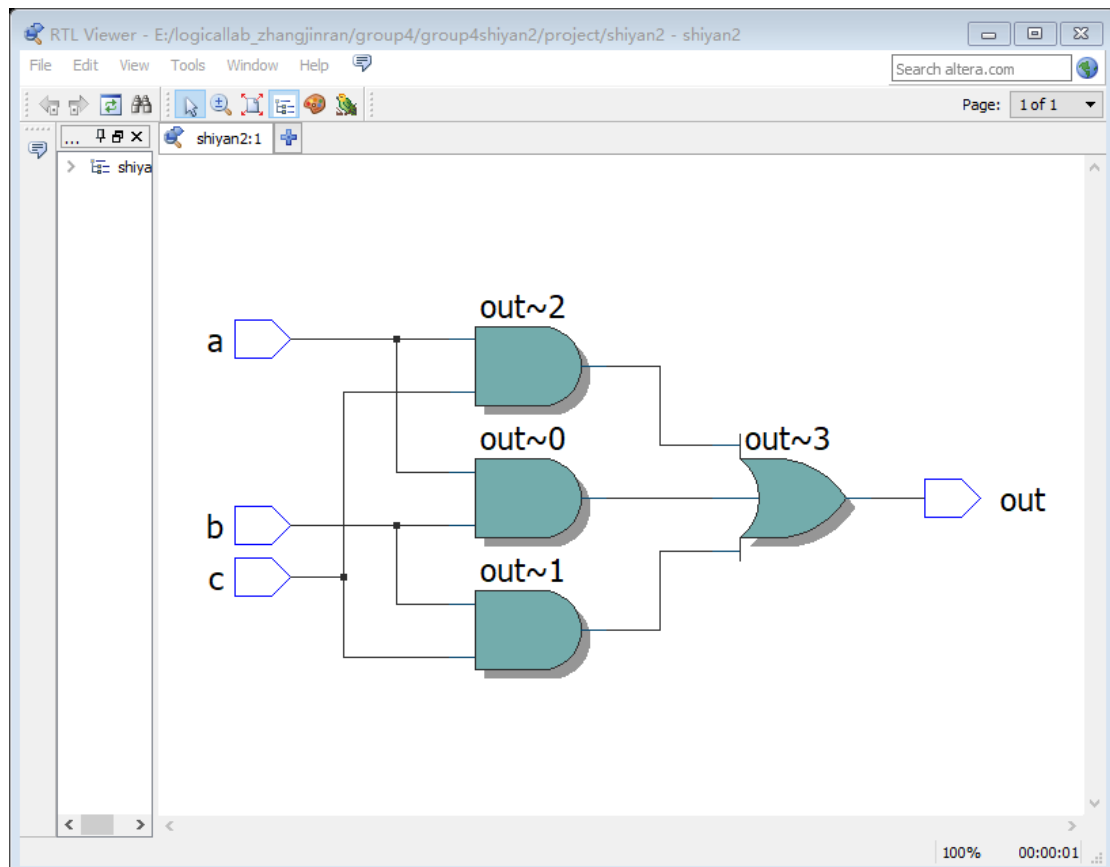
功能要求/工作原理：

实现 3 选 2 表决器。即多数表决器。a, b, c 三个输入，out 一个输出。 $out = ab + bc + ac$ 。三个输入中有两个或三个为 1 时，输出为 1，否则，输出为 0。

调试步骤及过程说明：

三个输入，两个输出，三个二输入与门，一个三输入或门。

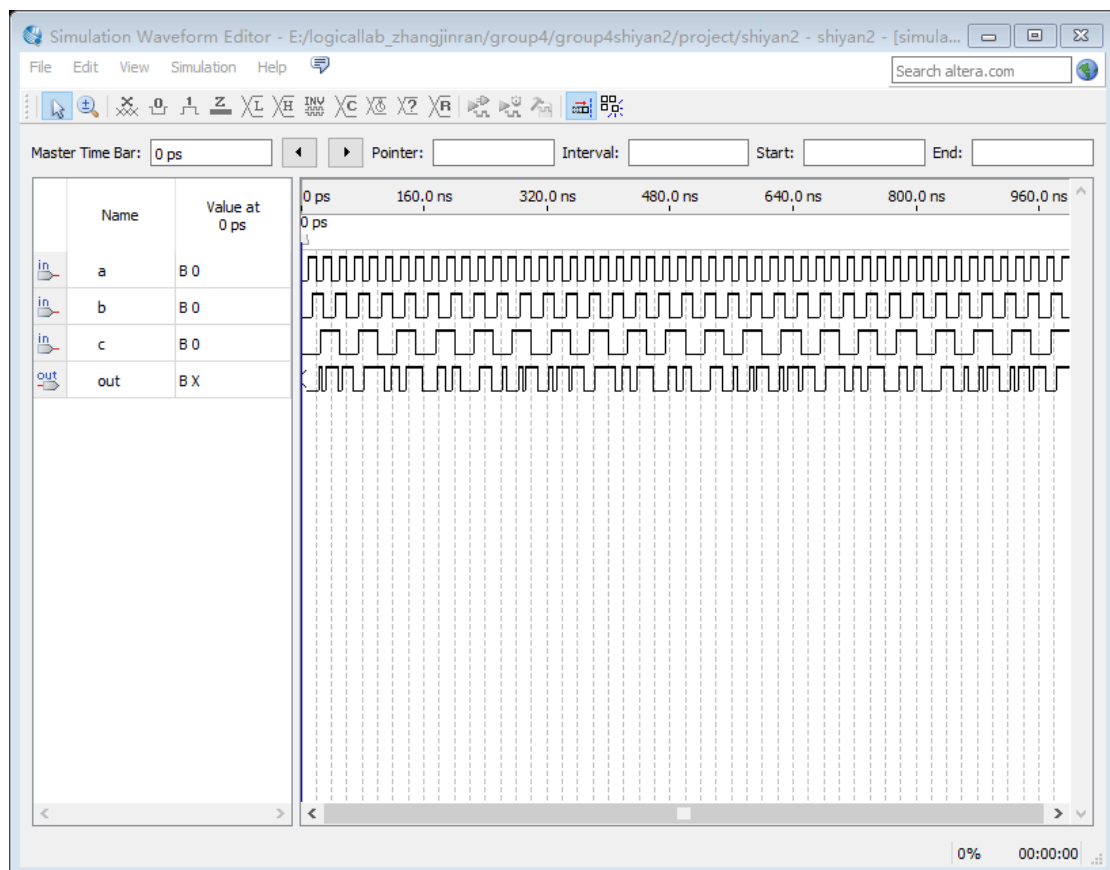
out: 三个输入两两进行与运算的结果再进行或运算



代码清单：

```
module shiyan2(  
    input wire a,  
    input wire b,  
    input wire c,  
    output wire out  
);  
assign out = (a&b)|(b&c)|(c&a);  
endmodule
```

仿真结果截图：



实验结果分析及其他问题说明：

三个输入中，所有情况都有两个或三个输入的值相同。此时，选择这个值做为输出的值。即实现多数表决。

三个输入中有两个或三个为 1 时，输出为 1，否则，输出为 0。

实验 3——4 位比较器

功能要求/工作原理：

对于四位比较器，输入 $2 \times 4 = 8$ 路，输出 3 路。命名输入为：A[3]、A[2]、A[1]、A[0]、B[3]、B[2]、B[1]、B[0]。如果：A>B，Bigger 输出为 1，其它输出为 0；A=B，Equal 输出为 1，其他输出为 0；A<B，Smaller 输出为 1，其他输出为 0。

给出公式：

$$\text{Bigger} = (A[3] \oplus B[3])A[3] + (A[3] \odot B[3]) (A[2] \oplus B[2])A[2] + (A[3] \odot B[3]) (A[2] \odot B[2]) (A[1] \oplus B[1])A[1] + (A[3] \odot B[3]) (A[2] \odot B[2]) (A[1] \odot B[1]) (A[0] \oplus B[0])A[0]$$

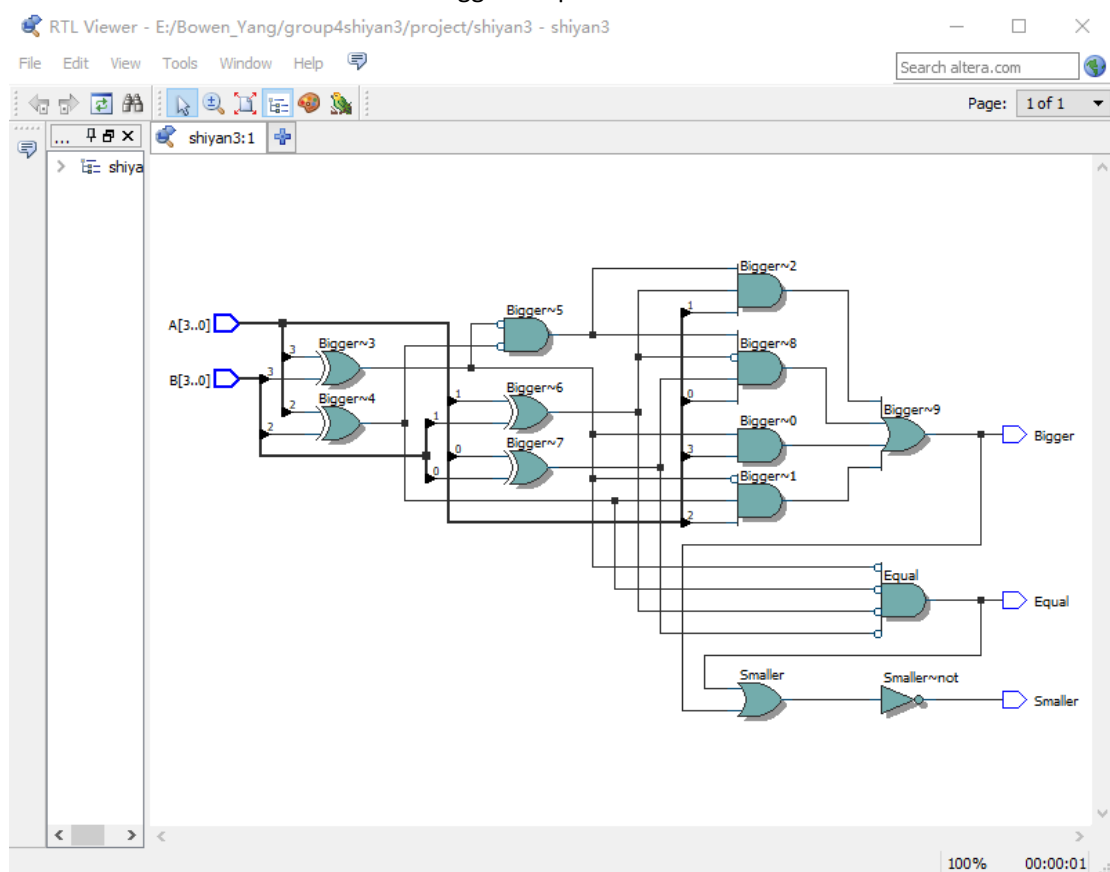
$$\text{Equal} = (A[3] \odot B[3]) (A[2] \odot B[2]) (A[1] \odot B[1]) (A[0] \odot B[0])$$

$$\text{Smaller} = \sim(\text{Bigger} \mid \text{Equal})$$

上述公式是在前位相同的情况下比较后位。如： $(A[3] \oplus B[3])A[3]$ 要求 A[3]=1 and B[3]=0。

调试步骤及过程说明：

利用 4 位输入，然后分别给出 Bigger、Equal、Smaller 值的计算即可。



代码清单：

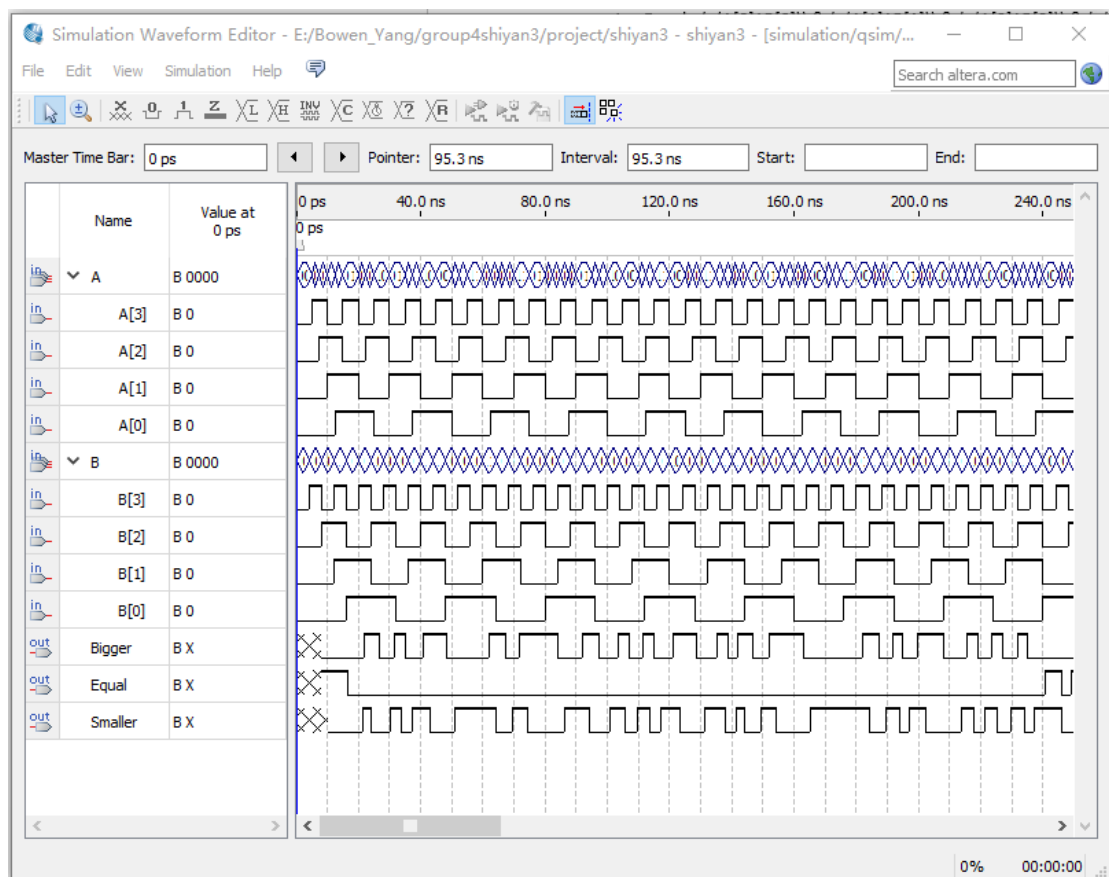
```
module shiyan3
(
    input wire[3:0] A,
    input wire[3:0] B,
    output wire Bigger,//A>B
    output wire Equal,//A==B
    output wire Smaller

);

assign Bigger=((A[3]^B[3])&A[3] | ~(A[3]^B[3]) & ((A[2]^B[2])&A[2])) | ~(A[3]^B[3]) &
(~(A[2]^B[2])) & ((A[1]^B[1])&A[1])) | ~(A[3]^B[3]) & ~(A[2]^B[2])) & ~(A[1]^B[1])) &
((A[0]^B[0])&A[0]));
assign Equal= ~(A[0]^B[0])) & ~(A[1]^B[1])) & ~(A[2]^B[2])) & ~(A[3]^B[3]));
assign Smaller=~(Bigger | Equal);

endmodule
```

仿真结果截图：



实验结果分析及其他问题说明：

利用不同的时间间隔组合 A、B；然后比较。

打完代码之后想到可以直接通过自带的比较符号来进行比较。不需要用这么麻烦的公式。

实验 4——常量的选择输出

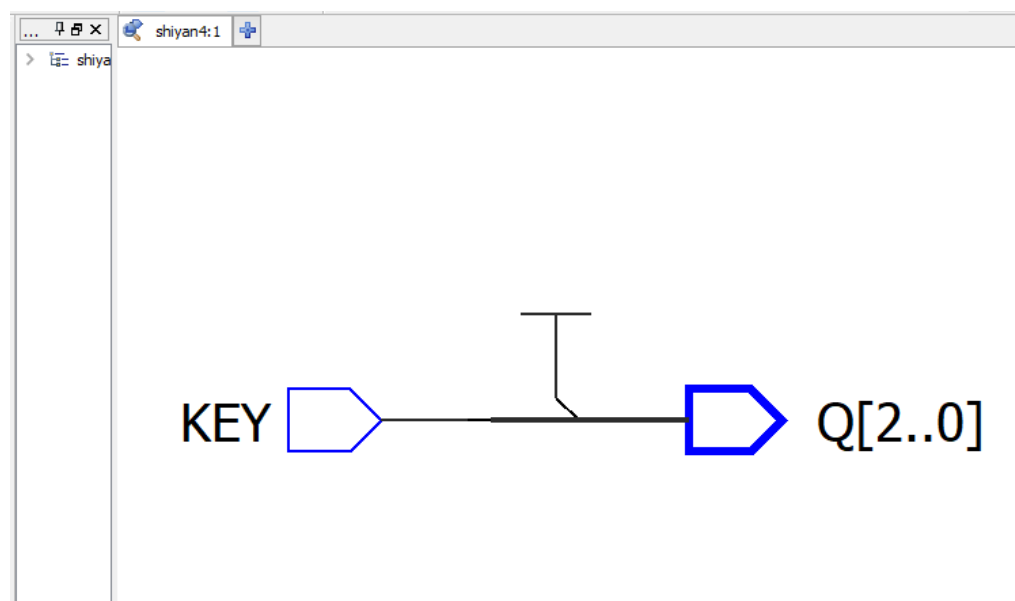
功能要求/工作原理：

输入 1 路（KEY），输出 1*3=3 路。对于 KEY=1 来说，输出常量 C1=111；对于 KEY=0 来说，输出常量 C2=010。

建立常量 C1、C2，然后输入 KEY 输出 Q，判断 KEY 的值以改变 Q。利用 if、else 实现即可。

另外，也可以通过真值表和公式来实现，也就是： $Q[2]=Q[0]=KEY, Q[1]=1$ 。但这样很显然没有用到应该用的常量，所以不这样写。

调试步骤及过程说明：



代码清单：

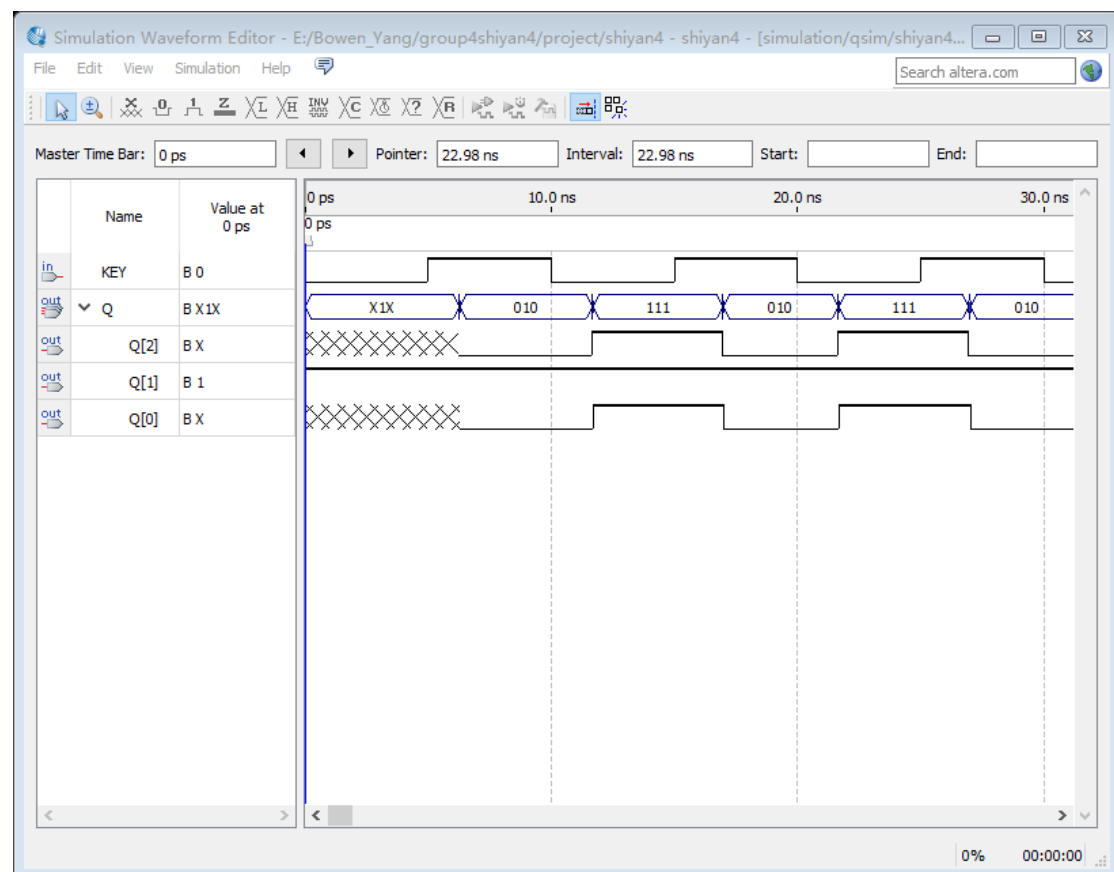
```
module shiyan4
(
    input wire KEY,
    output reg[2:0] Q
```

```
);
localparam [2:0] C1=3'b111;
localparam [2:0] C2=3'b010;
```

```
always @(KEY) begin
    if (KEY) begin
        Q=C1;
    end
    else begin
        Q=C2;
    end
end
```

```
endmodule
```

仿真结果截图：



实验结果分析及其他问题说明：

在这张图中，所求的基本功能均实现。

有一些问题是 Q[1]恒等于 1，为什么在其他门还没输出结果的时候就已经输出 1 了呢？

这好像不太适合功能，但也没想到更好的。